

Performance Comparison of Service Mesh Frameworks: the mTLS Test Case

Anat Bremler Barr

Tel Aviv University

Tel Aviv, Israel

anatbr@tauex.tau.ac.il

Ofek Lavi

Tel Aviv University

Tel Aviv, Israel

ofeklavi@mail.tau.ac.il

Yaniv Naor

Reichman University

Herzliya, Israel

yanivnaor92@gmail.com

Sanjeev Rampal

Red Hat

Raleigh, USA

srampal@redhat.com

Jhonatan Tavori

Tel Aviv University

Tel Aviv, Israel

jhonatan.tavori@cs.tau.ac.il

Abstract—Service Mesh has become essential for modern cloud-native applications by abstracting communication between microservices and providing zero-trust security, observability, and advanced traffic control without requiring code changes. This allows developers to leverage new network capabilities and focus on application logic without managing network complexities. However, the additional layer can significantly impact system performance, latency, and resource consumption, posing challenges for cloud managers and operators.

In this work, we investigate the impact of the mTLS protocol—a common security and authentication mechanism—on application performance within service meshes. Recognizing that security is a primary motivation for deploying a service mesh, we evaluated the performance overhead introduced by leading service meshes: *Istio*, *Istio Ambient*, *Linkerd*, and *Cilium*. Our experiments were conducted by testing their performance in service-to-service communications within a *Kubernetes* cluster.

Our experiments reveal significant performance differences (in terms of latency and resource consumption) among the service meshes, rooting from the different architecture of the service mesh, sidecar versus sidecarless, and default extra features hidden in the mTLS implementation. Our results highlight the understanding of the service mesh architecture and its impact on performance.

Index Terms—cloud, Service Mesh, Kubernetes, mTLS, eBPF

I. INTRODUCTION

A service mesh is a dedicated infrastructure which controls service-to-service communication over a network and provides a way to control how different parts of an application share data with one another. Service mesh is most commonly used on top of the *Kubernetes* platform to overcome the challenges of a large-scale microservice-based system. A survey of the Cloud Native Computing Foundation (CNCF) [1] found that 70% of the respondents run a service mesh in production or development and 19% are in evaluation.

The number of service mesh providers is constantly growing, with each offering different features, performance, ease of use, and pricing, while continuously expanding capabilities. Typically, third parties develop and maintain the service mesh layer, allowing application developers to focus on business logic without worrying about network complexities. As services work by responding to incoming requests and issuing outgoing requests, the flow of requests becomes a critical determining factor of how the application behaves at runtime. Thus, standardizing the management of this traffic is crucial for guaranteeing the application's runtime.

A CNCF survey [1] examined factors driving organizations to adopt service meshes. Security was in the top concerns, with 79% of respondents relying on techniques like mTLS to reduce attack risks. This aligns with the growing adoption of the zero-trust security model, which requires all system users to be authenticated before accessing data [2].

While service meshes offer benefits like improved reliability and consistency, their impact on application performance remains a concern for software architects. The performance overhead introduced by service meshes, typically assessed through metrics like throughput, latency, and resource consumption (CPU and memory), arises from the additional hops required for each system call. This overhead can increase request and response times, potentially reducing the overall benefits that service meshes are designed to offer.

The goal of this work is to compare the performance of four common service mesh models—Istio [3], Istio Ambient [4], Linkerd[5], and Cilium [6]—in terms of latency and resource usage. Further, we aim to study the following core questions: How does the mTLS protocol affect network performance? What is the performance cost of offloading mTLS logic to the service mesh? We seek to offer a comprehensive assessment of the trade-offs involved in using service meshes in performance-critical environments.

To evaluate the service mesh frameworks, we set up a testing environment that simulates a production-like cloud environment. During testing, we monitor the system's behavior under load through standard monitoring tools such as Prometheus[7].

Table I summarizes the main difference between the service-mesh architectures characteristics and the main experiment results (explained in detailed in Sections II and V respectively). We included GitHub star counts as an indication of popularity.

Service Mesh	Model	Proxy	GitHub Stars	P99 Latency (seconds)	CPU (cores)	Memory (MiB) ¹
Baseline	-	-	-	0.22	client: 0.08 server: 0.12	client: 152 server: 71
Istio	Sidecar	Envoy	35.9k	+0.38	client: +0.81 server: +0.87	client: +255 server: +169
Istio Ambient	Sidecarless (node agent)	Ztunnel	35.9k	+0.02	client: +0.23 server: +0.23	client: +26 server: +26
Linkerd	Sidecar	Linkerd2 -proxy	10.6k	+0.09	client: +0.29 server: +0.22	client: +62 server: +63
Cilium	Sidecarless (node agent)	Envoy	20.1k	+0.22	client: +0.12 server: +0.08	client: +95 server: +95

TABLE I: Comparison of Service Mesh Models Performance (3200 RPS, 1600 connections, intra-node communication). The order of preference is: green > yellow > orange > red. The numbers are relatively (+) to the baseline.

Our experiments reveal significant performance differences among the service meshes. Enforcing mTLS increased latency across all tested providers, with increases of 166% for Istio, 8% for Istio Ambient, 33% for Linkerd, and 99% for Cilium. In some tests, Istio’s latency increase was almost four times that of Linkerd and more than 6 times that of Istio Ambient. Furthermore, our results highlight the performance benefits of a sidecarless architecture and the use of eBPF. To understand the root cause of Istio’s high latency, we discovered that specific steps in request processing, such as HTTP parsing, which were part of the default mTLS implementation, significantly contribute to the overall performance overhead.

Our findings can be used by decision-makers to select the most appropriate service mesh provider and architecture for their applications.

II. BACKGROUND

A. Kubernetes: Containers, Pods, Nodes and Clusters

Kubernetes is a widely-used open-source system for automating the deployment, scaling, and management of applications [8]. Containers, Pods, Nodes, and Clusters are the core components of Kubernetes. *Containers* are self-contained environments for applications, providing repeatability across diverse cloud and OS infrastructures by decoupling applications from the host infrastructure. A *Pod*, the smallest deployable unit in Kubernetes, can host one or more containers that share storage and network resources. *Nodes*, either physical or virtual machines, provide the infrastructure for running Pods and are managed by the Kubernetes control plane. A cluster consists of a control plane with a set of worker nodes. *Clusters* ensure fault-tolerance and high availability by distributing workloads and managing resources across the system. Service meshes are built on these Kubernetes components to manage the communication between services within the cluster.

B. Service Mesh: Architectures

There are two main models for managing network traffic in a service mesh: the sidecar pattern and the sidecarless model. The sidecar pattern deploys application components in separate containers for isolation and encapsulation. See Figure 1(a). Attached to the main application container, the sidecar shares its lifecycle, starting and stopping with the parent. The sidecar proxy manages all incoming and outgoing network traffic for a service – handling flow, gathering telemetry, and enforcing policies. The service itself remains unaware of the network, relying on the sidecar. In Kubernetes, the sidecar proxy is an additional container automatically injected into each Pod, managing the main container’s network traffic.

In the sidecarless model, the proxy container is moved to the host and kernel, eliminating the need for a sidecar for each pod. This is enabled by eBPF (extended Berkeley Packet Filter) [9]. See Figure 1(b). An eBPF-based implementation can redirect packets using eBPF programs hooked to specific kernel points. A single agent on each node manages eBPF

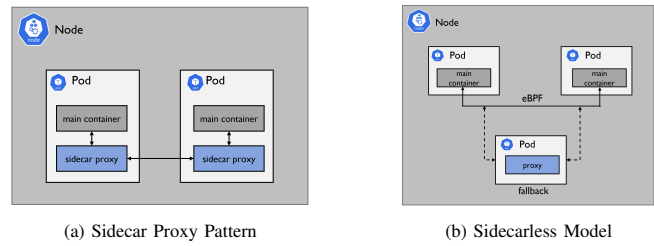


Fig. 1: Service Mesh Architectural Models

programs, allowing it to control network in and out of containers using the Linux kernel. Features like L3/L4 forwarding, routing, and network policies are handled directly in eBPF, while tasks like connection splicing, rate limiting, or TLS termination fall to the Node’s central agent for L7 processing, when eBPF is not capable of processing the request.

We chose the tested providers based on their popularity and maturity, ensuring that the findings of this study are relevant to a wide audience. To better investigate performance factors, we chose service meshes with both similarities and differences. Below, we briefly highlight the key features and differences of the studied service meshes.

Istio [3]: A widely adopted open-source service mesh built on Envoy [10]. It has a data plane with Envoy-based sidecar proxies and a control plane that manages the proxies and traffic routing. It was the first Kubernetes-native solution with advanced features like comprehensive analytics.

Istio Ambient [4]: Istio Ambient is a newer, sidecar-less architecture for Istio that aims to reduce the complexity and resources overhead associated with traditional sidecar proxies by using lightweight node-level proxies, utilizing eBPF.

Linkerd [5]: An open-source Kubernetes service mesh under the CNCF, licensed under Apache v2. Architecturally similar to Istio, Linkerd employs a data plane and a control plane, deploying transparent proxies within each Pod to auto-manage traffic. Unlike Istio, Linkerd uses its own lightweight micro-proxy, linkerd2-proxy, instead of Envoy.

Cilium [6]: An open-source project for Kubernetes clusters and other container orchestration platforms. Unlike Istio and Linkerd, Cilium adopts a unique service mesh architecture by allowing the mesh to operate entirely without sidecars. Instead of deploying proxy containers with each pod, it relocates the functionality to the host and kernel, utilizing eBPF.

C. mTLS: Network Security

Transport Layer Security (TLS) is a widely used security protocol that ensures privacy and data integrity. Proposed by the Internet Engineering Task Force (IETF) in 1999, the latest version, TLS 1.3, was released in 2018 [11]. It is typically layered atop the Transmission Control Protocol (TCP) to secure application protocols like HTTP but can also work with UDP. TLS uses asymmetric cryptography to verify the server’s identity and symmetric key cryptography to encrypt the data exchanged between client and server, preventing man-in-the-middle attacks and keeping intercepted messages unreadable.

¹ 1 MiB (Mebibyte) is equal to 2^{20} bytes

Mutual Transport Layer Security (mTLS) is very similar to the TLS protocol. While in TLS, the client verifies the server's identity by asking for and validating its certificate. In mTLS there is an additional step involved in which the server also asks for the client's certificate and verifies it at their end. A secure connection for data transfer is only established when both client and server successfully authenticate themselves and verify each other's certificates. As companies shift to zero-trust security², mTLS is becoming very popular, providing secure communication policies between microservices.

III. RELATED WORK

Performance evaluations of service mesh have been conducted by the providers themselves: Istio [13], Linkerd [14], Cilium [15], and [16], [17], [18]. Repeating these tests from a third-party perspective is crucial to validate their results and offer deeper insights. Several studies have explored the performance overhead of service meshes. [19] developed MeshInsight, profiling components and predicting overhead based on configuration and workload. [20] examined Istio's traffic management features (e.g., circuit breakers and retries) impact on performance through experiments with varying configurations. [21] found significant performance impacts of service meshes in edge environments. Recent benchmarks [22] evaluated a sidecar-free model (Cilium), showing significant latency improvements compared to other service meshes by avoiding the overhead of running two proxies between connections. [23] focused on performance-demanding workloads and concluded that current service mesh architectures are inadequate, calling for research into more efficient designs and improved quality of service.

The performance impact of TLS has been studied extensively. In 1999 [24] found that the overhead due to TLS can decrease the number of HTTP transactions handled by up to two orders of magnitude. In 2018, [25] evaluated the additional overhead of mutual authentication instead of just server-side authentication and showed that its further effect is minimal both in latency and throughput, either with or without session reuse. [25] stated that TLS may only incur 12-40% penalty degradation thanks to session reuse. [26] found Istio's Ingress Gateway more performance efficient than Kubernetes' Load Balancer in a Zero-Trust multi-cloud environment.

IV. METHODOLOGY

Our methodology involves comparing native Kubernetes (referred to as the baseline), Kubernetes with mTLS (referred to as the baseline with mTLS), and a specific service mesh technology with mTLS. Our experiments are limited to a single cluster, but we compare inter-node (communication between different nodes in the cluster) and intra-node (communication between pods within the same node) scenarios.

²The zero-trust security model is based on guidelines established by the National Institute of Standards & Technology (NIST). The NIST 800-207 [12] publication defines the standard for zero-trust practices. In this model, all users, whether internal or external, must undergo continuous authentication and verification before accessing data.

We designed a representative microservices testing environment on an OpenShift Kubernetes Cluster hosted on GCP VMs (see specification at Table II). The cluster consisted of three node groups: *Control plane* (Kubernetes system pods), *System* (OpenShift components, monitoring tools, etc), and *Workload* (load generator, server). Separating these nodes isolated the testing subjects and minimized interference.

Machine type	n2d-highcpu-8
Kubernetes Version	1.24.6
Node CPU	8 vCPU
Node Memory	8 GB
OS	Fedora CoreOS 36
Kernel Version	6.08-200.fc36.x86_64
Container Runtime	cri-o/1.24.3

TABLE II: Node Specifications

For the workload generation, we utilized Fortio [27] to generate simulated network traffic. Fortio operates at a specified RPS (Requests Per Second), records execution time histograms, and calculates percentiles. For simulating a server, we implemented a simple HTTP server in Go using the `net/http` library [28]. To simulate server processing time, we added a 200ms delay before returning responses.

To evaluate performance overhead, we focused on *Latency*, *CPU Usage*, *Memory Usage*, and *Error Rate*. Fortio reports latency metrics exclusively in the form of percentiles. Therefore, we analyzed the 99th and 50th percentile latencies in this study. As the trend from the results from both percentiles were nearly identical, we chose to focus solely on the 99th percentile in this paper. We show in the Figures only the server side performance however the client side exhibits a similar trend. For the complete set of results (with client-side measurement, and 50th percentile) and exact configuration flags see our technical report [29].

For service meshes that have the sidecar pattern approach, we enabled the automatic mechanism for injecting the sidecar proxy. In addition, we configured each service mesh to enable or disable mutual TLS (mTLS) as required. In Cilium, the configuration enables IPsec or WireGuard for the inter-node communication and there is no authentication in the intra-node communication. For identity management, we deployed the SPIRE server.

A. Test Scenarios

We assessed each service mesh's performance overhead by conducting tests starting with a baseline without any service mesh, then repeating the tests under identical conditions with each service mesh installed to quantify the overhead. Tests were executed at constant request rates, for 5 minutes each, and repeated multiple times to ensure result stability, indicated by low variance. Source code is available on GitHub [30].

We designed several test scenarios to evaluate the performance impact of different service mesh configurations.

1) *Baseline scenario*: The baseline test measured the system's performance without any service mesh, serving as a reference point for subsequent comparisons. Traffic routing relied on Kubernetes' native networking with `kube-proxy`[31].

2) *Baseline mTLS scenario*: This test is similar to the baseline test but with mTLS enforced between the client and server using Go's `crypto/tls` library [32]. Certificates were generated using OpenSSL [33].

3) *Sidecar Pattern scenario*: In this scenario, relevant to Istio and Linkerd, we injected sidecar proxies into the pods, and configured them to enforce mTLS communication.

4) *Sidecarless Pattern scenario*: For Cilium and Istio Ambient, we evaluated the sidecarless pattern, where authentication and encryption are handled by a per-node agent without sidecar proxies. The proxies were configured to enforce mTLS.

V. EXPERIMENTAL RESULTS

A. Baseline Test: mTLS impact

Prior to analyzing the impact of Service Mesh implementations, we first established a baseline to understand the effect of mTLS on performance in the absence of a service mesh. We conducted 5-minute tests with 160, 1600, and 6400 concurrent connections at 320, 3200, and 12,800 RPS, respectively (2 RPS for each connection).

Figure 2 illustrates the latency from the baseline test compared to the baseline-mTLS test.

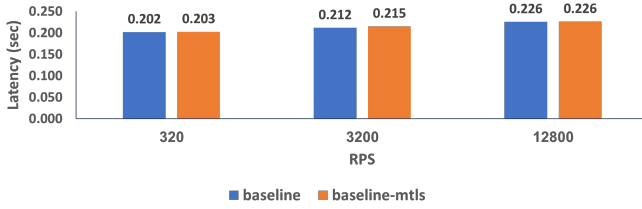


Fig. 2: Baseline tests Latency

The latency increase in the baseline-mTLS test is relatively low. There is up to 3% increase in the 99th percentile. Even under high load conditions (larger number of connections), the latency overhead introduced by mTLS was relatively small. The results for the 50th percentile were similar.

However, Fig 3, shows that enabling mTLS led to a notable increase in resource usage, with CPU and memory consumption increasing by up to 100% in client pods (and similarly in server pods - see technical report[29]).

B. Service Mesh : mTLS impact

In each test, we installed the selected service mesh in the cluster and enforced using mTLS by the service mesh and conducted 5-minute tests with 160, 1600, and 6400 concurrent connections at 320, 3200, and 12,800 RPS, respectively (2 RPS for each connection).

As can be seen in Fig 4, a significant latency increase was measured in all service meshes with high variance between them. Istio had the highest latency increase, and Istio Ambient had the lowest latency. Moreover, as the number of RPS increases, the difference between the service meshes grows as well. In the test with the highest load (12,800 RPS), the load generator could not even achieve the target RPS with the Istio sidecar due to the high latency (achieved 6868 RPS).

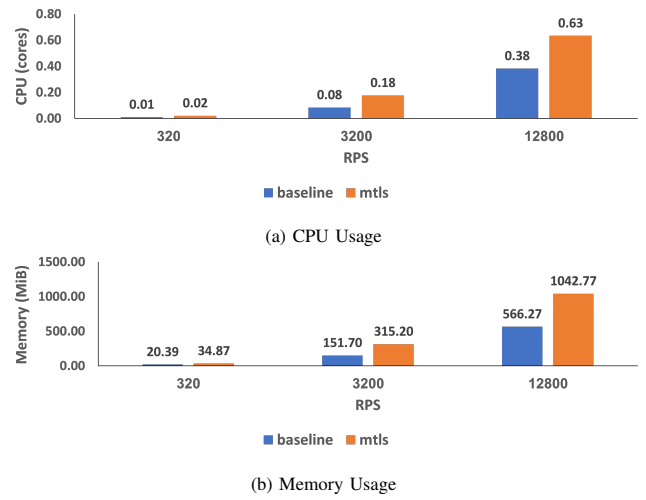


Fig. 3: Baseline Resource Tests

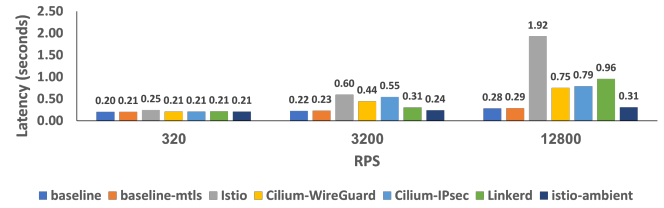


Fig. 4: Latency as function of load

Fig 5 shows that Cilium and Istio-Ambient had the best performance in terms of CPU consumption and memory consumption respectively, while Istio was the worst in terms of CPU and memory consumption.

Fig. 6 shows the latency difference between inter-node communication (communication between different nodes) and intra-node communication (communication within the same node). It demonstrates a notable latency difference for Cilium, while Istio and Linkerd yielded similar results. While mTLS was enabled in all cases, Cilium, unlike Istio and Linkerd, disables network encryption for intra-node traffic by design. This behavior, consistent across both WireGuard and IPsec configurations, leads to security implications but with a significant performance improvement.

C. Impact of Configuration on Istio Throughput

In this test, we focus on Istio, which shows the highest latency and memory consumption with the number of connections compared to the baseline. Our goal is to better understand the impact of different configurations on the observed performance. We tested throughput on several scenarios, where in each scenario, the load generator was configured to achieve maximum RPS, initiating new requests immediately after previous ones were completed. We check the impact of enabling the combination of three default features in Istio: (1) mTLS, (2) Metrics - telemetry collection and (3) Http parsing.

The scenarios we tested are: (i) *mTLS* - mTLS is enabled with Metrics and Http parsing. (ii) *Plaintext* - mTLS is

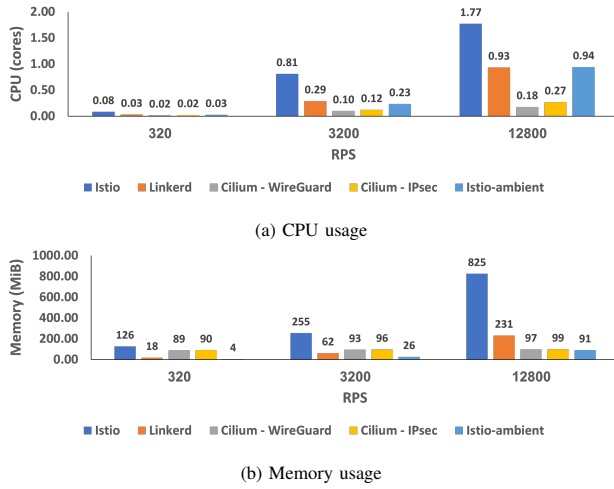


Fig. 5: Service Mesh's proxies CPU and Memory Usage³

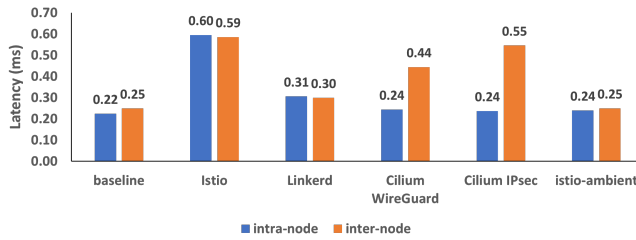


Fig. 6: Inter-node vs Intra-node communication latency

disabled but Metrics and Http parsing are enabled. (iii) *no-Metrics* - mTLS, Metrics are disabled and Http parsing is enabled, (iv) *TCP* - mTLS and HTTP Parsing are disabled, but Metrics is enabled. (v) *TCP-no-Metrics* - all features are disabled: mTLS and HTTP Parsing and Metrics are disabled.

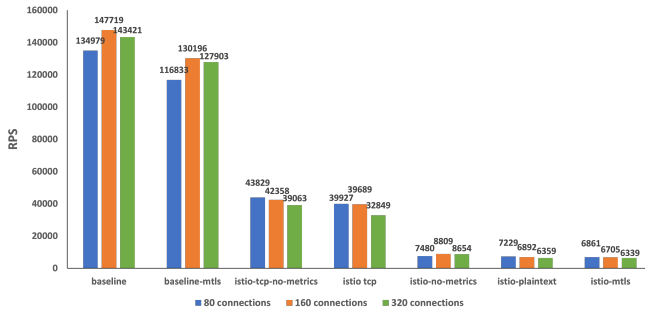


Fig. 7: Istio throughput with different configurations comparing to the baseline

As shown in Fig 7 adding Istio's sidecar with mTLS caused a 95% decrease in throughput compared to the baseline (Kubernetes without Istio). Disabling mTLS produced similar results, indicating that the high latency is not due to the mTLS enforcement but due to the sidecar and extra functionalities. Disabling metrics collection only slightly improved throughput. Istio-tcp, where HTTP parsing was also disabled,

³It represents sidecar proxies' resources in Istio and Linkerd, and the central node agent's resources in Cilium and Istio Ambient.

increased throughput nearly fivefold compared to Istio with only mTLS disabled, indicating the high resource consumption of HTTP parsing compared to mTLS and metrics collection.

D. Memory Consumption Analysis

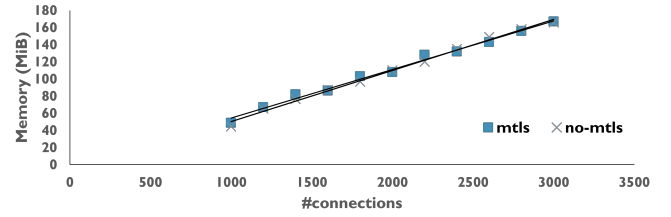


Fig. 8: Istio-ambient: Memory Consumption by Number of Connections. Measuring client-side with inter-node communication.

To further investigate the impact of memory consumption differences between service mesh implementations, we tested whether memory usage scales with the number of connections and if it is impacted by the mTLS. We chose to focus on Istio Ambient this time, since it gave the best memory results. In this experiment, we maintained a constant request rate of 3200 RPS while increasing concurrent connections from 1000 to 3200. As shown in Fig. 8, memory consumption increased linearly with the number of connections, indicating each connection adds a constant memory overhead. The coefficient of determination (R^2) was 0.99, confirming a strong linear relationship. The results with the server and with intra-node communication were similar.

However, enabling mTLS did not show an increase in memory consumption and the two graphs of memory consumption with MTLS and without are similar as connections grew. This indicates that while connection count affects total memory usage, the overhead from mTLS remains relatively constant regardless of the number of connections.

E. Discussions

We summarize the findings of 3200 RPS in Table I. We add also detailed information about client-side and server-side from our detailed experiments in the technical report [29]. There is no correlation between popularity (GitHub stars, Table I) and performance, suggesting that developers prioritize features and community support over performance.

1) *Istio*: Istio exhibited the highest latency increase, with a 166% rise at 3,200 RPS, and failed to reach the target at 12,800 RPS due to its sidecar model. This architecture, which adds a proxy for each pod and extra network hops, also resulted in the highest CPU and memory overhead.

2) *Istio Ambient*: Istio Ambient showed the best latency performance, with only an 8% increase at 3,200 RPS and low latency even at 12,800 RPS, outperforming all the others. It was also the most memory-efficient, eliminating per-pod proxies. In terms of CPU usage, Istio Ambient improved significantly over traditional Istio and was similar to Linkerd, though less efficient than Cilium. Overall, Istio Ambient offers

a scalable solution with relatively low resource usage while maintaining a robust feature set.

3) *Linkerd*: In terms of latency, Linkerd performed moderately. It lagged behind Istio Ambient across all load levels, and behind Cilium at 12,800 RPS, indicating that while Linkerd is efficient, its sidecar-based architecture introduces more overhead than Istio Ambient's node-level model. For CPU consumption, Linkerd performed similarly to Istio Ambient, better than Istio but not as efficient as Cilium. In terms of memory consumption, Linkerd ranked second best, just behind Istio Ambient. However, in large-scale environments with many pods, Linkerd's sidecar model may increase resource consumption due to the need for proxy for each pod.

4) *Cilium*: In terms of latency, Cilium showed moderate performance at 3,200 RPS with 99% latency increase, while it outperformed Linkerd at 12,800 RPS, showing better efficiency with higher traffic. Cilium performed best in terms of CPU consumption, with its sidecarless model eliminating the need for a proxy per pod, making it highly scalable for large clusters. Its memory consumption was higher than that of Linkerd and Istio Ambient. Yet, in clusters with more pods per node, Cilium's memory efficiency could surpass Linkerd's, which might make it more favorable in such environments; this needs to be checked in future work. Note, however, that by design, Cilium does not encrypt intra-node traffic, which may be a necessary feature in some cases, but leads to significantly lower latency, reaching latency similar to Istio-Ambient.

5) *mTLS protocol Impact on network performance*: The baseline results show that mTLS has minimal impact on latency in Kubernetes service-to-service communication, but significantly increases CPU and memory usage. This suggests that resource overhead varies by implementation and programming language, and the increased resource consumption should not be overlooked. Additionally, the low latency impact may not apply universally across different scenarios or network topologies, and in the future work, cases where the client and server are in different clusters should be tested.

Our experiments show that shifting mTLS implementation to the service mesh significantly affects latency and resource consumption. The throughput test in the Istio example reveals this impact stems not from inefficient mTLS, but from adding a new network entity, with more functionalities, each contributing to performance overhead.

VI. CONCLUSIONS

The goal of this paper was to determine the best-performing service mesh in the mTLS case. However, the results reveal the complexity of evaluating performance and trade-offs, partly due to many hidden features and different functionality in the default settings of the different service mesh frameworks. Nonetheless, it is evident that a sidecarless architecture offers a performance advantage.

ACKNOWLEDGMENT

This research was partly supported by RedHat Research.

We are deeply grateful to Galia Shabtai for her insightful discussions. We honor her memory with heartfelt appreciation following her untimely passing.

REFERENCES

- [1] "Cncf service mesh survey 2022." [Online]. Available: https://www.cncf.io/wp-content/uploads/2022/05/CNCF_Service_Mesh_MicroSurvey_Final.pdf
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," 2020.
- [3] "Istio service mesh." [Online]. Available: <https://istio.io>
- [4] "Istio-ambient service mesh." [Online]. Available: <https://istio.io/latest/docs/ambient/>
- [5] "Linkerd service mesh." [Online]. Available: <https://linkerd.io/>
- [6] "Cilium service mesh." [Online]. Available: <https://cilium.io>
- [7] "Prometheus monitoring and alerting system." [Online]. Available: <https://prometheus.io>
- [8] "Kubernetes documentation." [Online]. Available: <https://kubernetes.io/docs/home/>
- [9] "ebpf." [Online]. Available: <https://ebpf.io/>
- [10] "Envoy proxy." [Online]. Available: <https://www.envoyproxy.io/>
- [11] "The transport layer security (tls) protocol version 1.3 (rfc)." [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8446>
- [12] "Nist zero trust architecture publication." [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/207/final>
- [13] "Istio performance and scalability." [Online]. Available: <https://istio.io/latest/docs/ops/deployment/performance-and-scalability>
- [14] "Linkerd benchmarks." [Online]. Available: <https://linkerd.io/2019/05/18/linkerd-benchmarks>
- [15] "Cilium performance evaluation." [Online]. Available: <https://docs.cilium.io/en/v1.9/operations/performance>
- [16] "Performance benchmark analysis of istio and linkerd." [Online]. Available: <https://kinvolk.io/blog/2019/05/performance-benchmark-analysis-of-istio-and-linkerd>
- [17] "Linkerd vs. istio." [Online]. Available: <https://buoyant.io/linkerd-vs-istio>
- [18] "Linkerd vs. istio benchmarks." [Online]. Available: <https://linkerd.io/2021/05/27/linkerd-vs-istio-benchmarks>
- [19] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting service mesh overheads," 2022. [Online]. Available: <https://arxiv.org/abs/2207.00592>
- [20] M. R. Saleh Sedghpour, C. Klein, and J. Tordsson, "An empirical study of service mesh traffic management policies for microservices," in *Proceedings of ACM/SPEC*, 2022, pp. 17–27.
- [21] M. Ganguli, S. Ranganath, S. Ravisundar, A. Layek, D. Ilangoan, and E. Verplanke, "Challenges and opportunities in performance benchmarking of service mesh for the edge," pp. 78–85, 2021.
- [22] Isovalent, "Cilium service mesh," 2022. [Online]. Available: <https://isovalent.com/blog/post/cilium-service-mesh>
- [23] A. O. Duque, C. Klein, J. Feng, X. Cai, B. Skubic, and E. Elmroth, "A qualitative evaluation of service mesh-based traffic management for mobile edge cloud," *arXiv preprint arXiv:2205.06057*, 2022.
- [24] G. Apostolopoulos, V. Peris, and D. Saha, "Transport layer security: how much does it really cost?" in *IEEE INFOCOM '99*, vol. 2, 1999.
- [25] P. Machnik, M. Niemiec, M. Urueña, and N. Stoianov, "Performance evaluation of indirect security architecture," *Iteckne*, vol. 15, p. 34–42, 06 2018. [Online]. Available: http://www.scielo.org.co/scielo.php?script=sci_arttext&pid=S1692-17982018000100034&nrm=iso
- [26] S. Rodigari, D. O'Shea, P. McCarthy, M. McCarry, and S. McSweeney, "Performance analysis of zero-trust multi-cloud," in *IEEE CLOUD*, 2021.
- [27] "Fortio testing tool." [Online]. Available: <https://fortio.org>
- [28] "Go net/http library." [Online]. Available: <https://pkg.go.dev/net/http>
- [29] "Technical report." [Online]. Available: <https://deepness-lab.org/publications/performance-comparison-of-service-mesh-frameworks-the-mtls-test-case>
- [30] "Source code and scripts." [Online]. Available: <https://github.com/flikhamud45/sm-performance>
- [31] "Kubernetes kube proxy." [Online]. Available: <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy>
- [32] "Go crypto/tls library." [Online]. Available: <https://pkg.go.dev/crypto/tls>
- [33] "Openssl." [Online]. Available: <https://www.openssl.org/>